UNIVERSIDAD DE LA SALLE
ESCUELA DE CIENCIAS BÁSICAS
PROGRAMA DE CIENCIA DE DATOS

Manual de Programación y Preubas - Televentas

Presentado por:
Sebastián Gutiérrez Guayacundo

Docente:
RODRIGO ARANDA FERNANDEZ

Lenguaje de Programación II
Semestre III – 2026
Bogotá, Colombia

Contenido

1. Enunciado del Problema: 3
2. Problema Resumido para el Código 3
3. Mapa UML de Código 4
4. Creación de Ramas y Repositorio 5
5. Algoritmo en Python (Lógica de Solución) 6
6. Código en Visual Studio Code 9
7. Prueba de Escritorio 11
Inicialización de Objetos (Memoria/Inventario) 11
Flujo del Cliente (Orden de Compra) 11
Flujo Interno (Logística y Operaciones) 11
Salida en Consola 12
Resultado: 12

1. Enunciado del Problema:

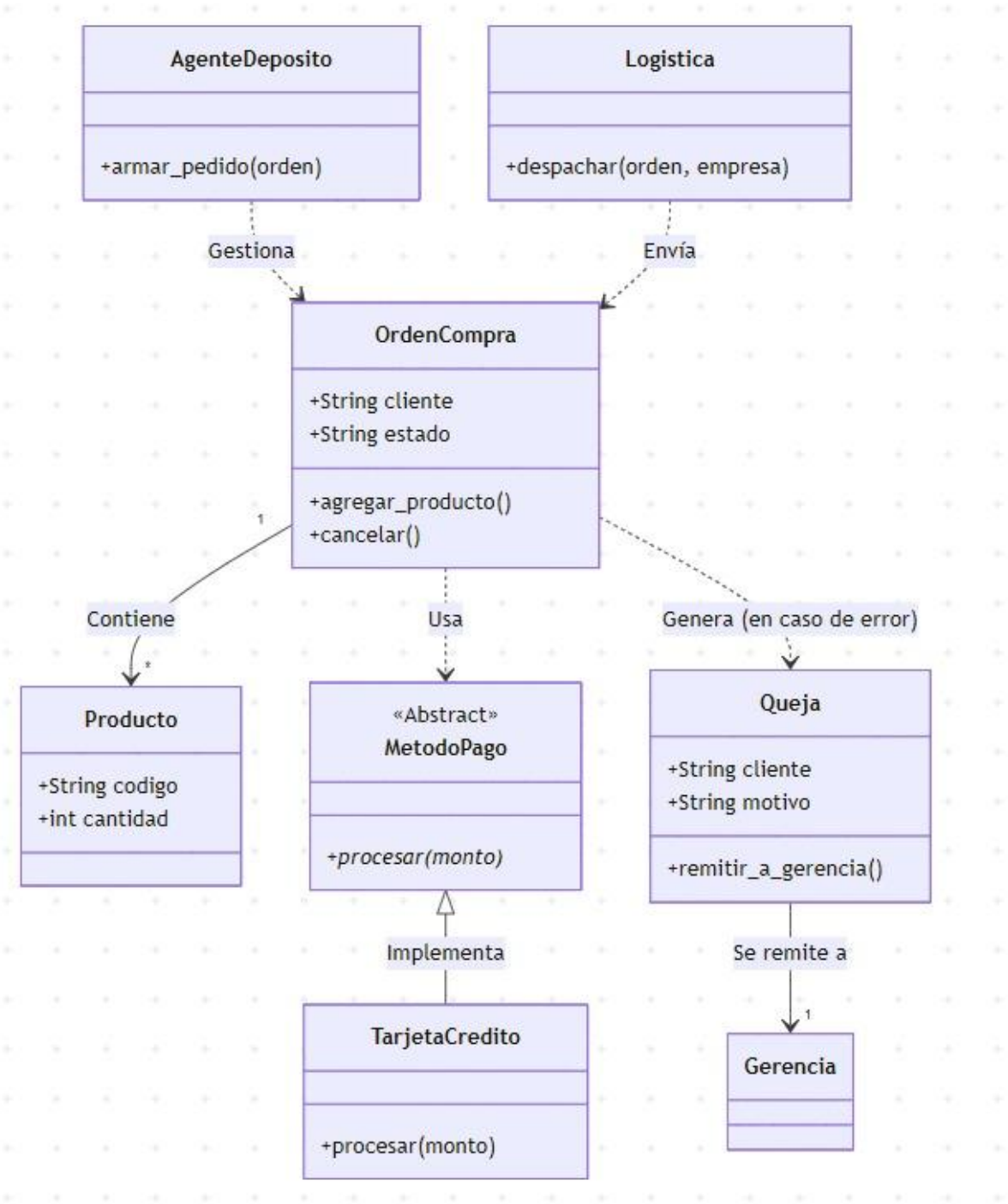
La empresa TeleVentas desea desarrollar un sistema web para el soporte de compras a distancia, de los productos que la empresa ofrece, por parte de sus clientes. Se prevé que, con este sistema, los clientes podrán, fundamentalmente: realizar consultas del catálogo de los productos que se ofrecen, para obtener información acerca de un producto particular (código, descripción, precio, cantidad disponible, etc.); solicitar le envíen periódicamente y por correo electrónico dicho catálogo de productos; ingresar una orden de compra, para que se le envíe un conjunto de productos, ajustándose a un tipo de pago para la misma (actualmente sólo tarjeta de crédito); presentar una queja (por demoras en la entrega de los productos, por ejemplo); y cancelar una orden.

Adicionalmente, se pretende que los agentes del depósito de la empresa utilicen el nuevo sistema, consultando las órdenes de compra confirmadas, para armar y empaquetar los productos pedidos en dichas órdenes de compra. Asimismo, se debe proveer soporte a estos agentes para determinar la logística para la entrega de cada pedido armado, seleccionando una empresa de transporte apropiada, y delegando a ésta la entrega del pedido correspondiente. Por otra parte, es destacable que la empresa ya posee un sistema de inventario para el control de stock de productos. El nuevo sistema deberá interactuar con éste para, por ejemplo: consultar descripción y precio de productos al tomar órdenes de compra, o bien para actualizar la disponibilidad de productos al armar pedidos. Por otra parte, las quejas recibidas por parte de los clientes son remitidas inmediatamente al gerente de relaciones de la empresa.

2. Problema Resumido para el Código

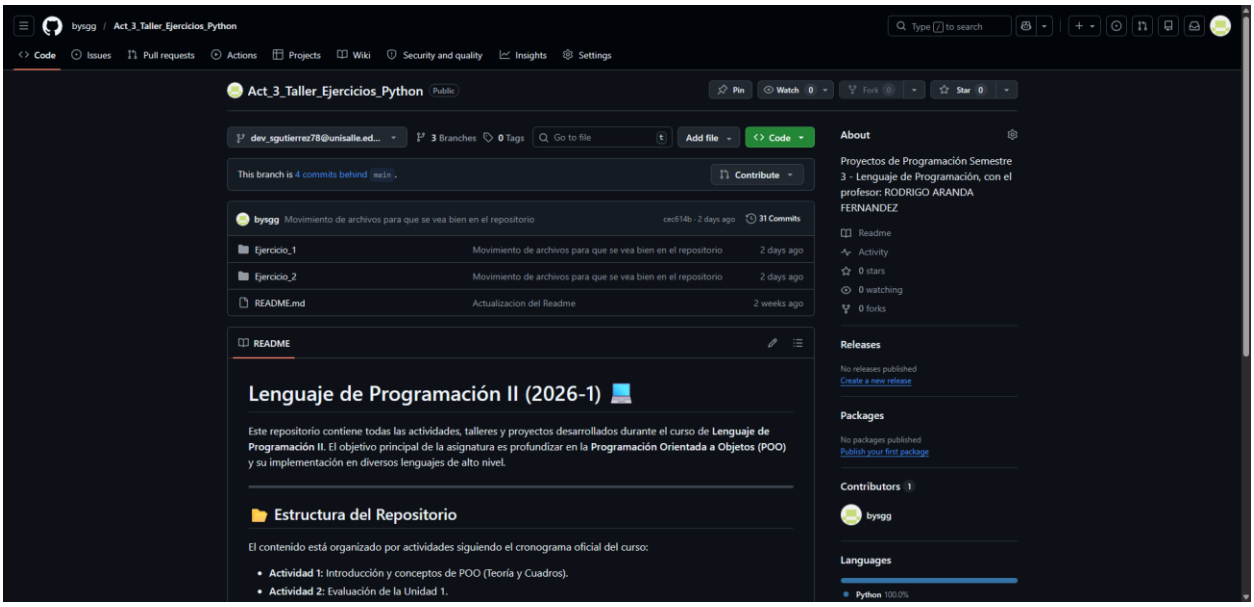
La empresa "TeleVentas" requiere un sistema robusto para gestionar ventas a distancia. El problema técnico principal radica en la sincronización de procesos distribuidos: un cliente inicia una orden, pero el stock debe apartarse inmediatamente para evitar ventas de productos inexistentes. Además, el flujo debe ser estrictamente secuencial; no se puede enviar un producto (Logística) si el depósito no lo ha empacado, y el depósito no puede empacar si la pasarela de pagos (Tarjeta de Crédito) no ha confirmado la transacción. Finalmente, el sistema debe ser capaz de desviar incidencias (Quejas) directamente a la alta gerencia sin interrumpir el flujo de ventas.

3. Mapa UML de Código



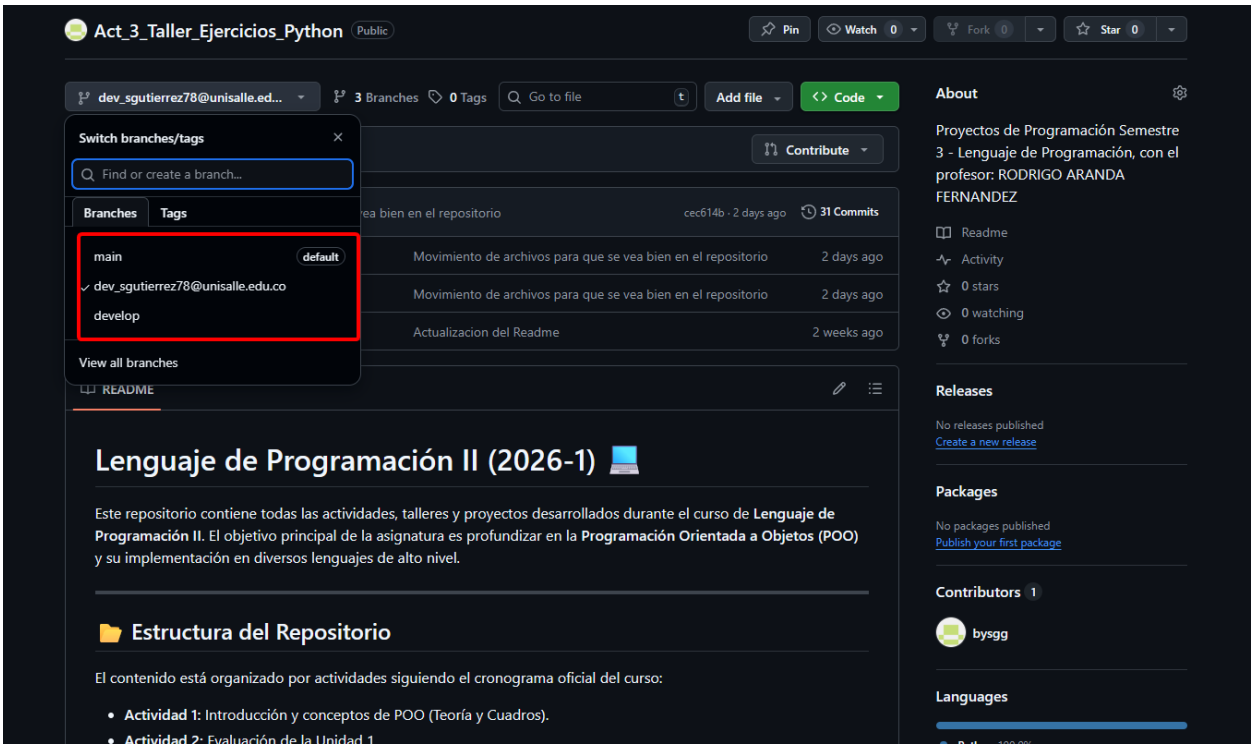
4. Creación de Ramas y Repositorio

https://github.com/bysgg/Act_3_Taller_Ejercicios_Python/tree/dev_sgutierrez78%40unisalle.edu.co



Se crean las ramas

- **Main, Develop y dev_sgutierrez78@unisalle.edu.co**



5. Algoritmo en Python (Lógica de Solución)

El sistema se diseñó bajo un modelo de Flujo de Estados:

1. Carga de Inventario: Creación de objetos Producto.
2. Validación de Disponibilidad: Antes de agregar a la OrdenCompra, se verifica Producto.cantidad > 0.
3. Procesamiento de Pago: Invocación de la clase abstracta MetodoPago.
4. Actualización de Estatus: Cambio de estado de PENDIENTE a PAGADA.
5. Despacho y Envío: Delegación de la orden a las clases AgenteDeposito y Logistica.
6. Cierre de Gestión: Gestión de objetos Queja para soporte post-venta.

main.py

```
from modelos import Producto, TarjetaCredito, OrdenCompra, AgenteDeposito, Logistica, Queja
from typing import List

def mostrar_encabezado(titulo: str):
    print("\n" + "="*50)
    print(f"      {titulo}")
    print("="*50)

def ejecutar_sistema_ventas():
    mostrar_encabezado("SISTEMA DE TELEVENTAS - COMPRAS A DISTANCIA")

    # 1. SIMULACIÓN DE INVENTARIO EXISTENTE (Requisito: El sistema posee un inventario)
    p1 = Producto("A101", "Laptop Gaming Pro", 1200.50, 10)
    p2 = Producto("B202", "Mouse Ergonómico inalámbrico", 45.00, 50)
    p3 = Producto("C303", "Teclado Mecánico RGB", 80.00, 5)

    inventario: List[Producto] = [p1, p2, p3]

    # --- FLUJO DE CLIENTE ---
    # 2. CONSULTA DE CATÁLOGO (Requisito: Obtener info de código, descripción, precio, stock)
    print("\n[CATÁLOGO DE PRODUCTOS]")
    for prod in inventario:
        print(prod)

    # 3. GENERAR ORDEN DE COMPRA (Requisito: Ingresar orden de compra)
    orden_sebas = OrdenCompra("Sebastian Gutierrez")
    print("\n[CLIENTE] Creando orden de compra...")
    orden_sebas.agregar_producto(p1, 1) # Compra 1 Laptop
    orden_sebas.agregar_producto(p3, 2) # Compra 2 Teclados

    # 4. PROCESAR PAGO (Requisito: Actualmente sólo tarjeta de crédito)
    pago = TarjetaCredito()
    monto_total = (1200.50 * 1) + (80.00 * 2)
    if pago.procesar(monto_total):
        orden_sebas.estado = "PAGADA"

    # --- FLUJO INTERNO EMPRESA ---
```

```
    # 5. AGENTE DE DEPÓSITO (Requisito: Armar y empaquetar órdenes confirmadas)
    deposito = AgenteDeposito()
    deposito.armar_pedido(orden_sebas)

    # 6. LOGÍSTICA (Requisito: Determinar logística y empresa de transporte)
    empresa_envio = "Servientrega"
    Logistica.despachar(orden_sebas, empresa_envio)

    # 7. GESTIÓN DE QUEJAS (Requisito: Remitir inmediatamente al gerente)
    print("\n" + "-"*30)
    reclamo = Queja("Sebastian Gutierrez", "El paquete llegó con la caja golpeada.")
    reclamo.remitir_a_gerencia()

    # --- RESUMEN FINAL ---
    mostrar_encabezado("RESUMEN DE OPERACIÓN")
    print(f"Cliente: {orden_sebas.cliente}")
    print(f"Estado Final de la Orden: {orden_sebas.estado}")
    print(f"Stock Actualizado de {p1.descripcion}: {p1.cantidad} unidades")

if __name__ == "__main__":
    ejecutar_sistema_ventas()
```

modelos.py

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any

class Producto:
    def __init__(self, codigo: str, descripcion: str, precio: float, cantidad: int):
        self.codigo = codigo
        self.descripcion = descripcion
        self.precio = precio
        self.cantidad = cantidad

    def __str__(self):
        return f"[{self.codigo}] {self.descripcion} - ${self.precio} (Stock: {self.cantidad})"

class MetodoPago(ABC):
    @abstractmethod
    def procesar(self, monto: float) -> bool: # Agregamos -> bool para que coincida
        pass

class TarjetaCredito(MetodoPago):
    def procesar(self, monto: float) -> bool: # Ahora Pylance está feliz
        print(f"[Pago] Procesando ${monto} con Tarjeta de Crédito... ¡Exitoso!")
        return True

class OrdenCompra:
    def __init__(self, cliente: str):
```

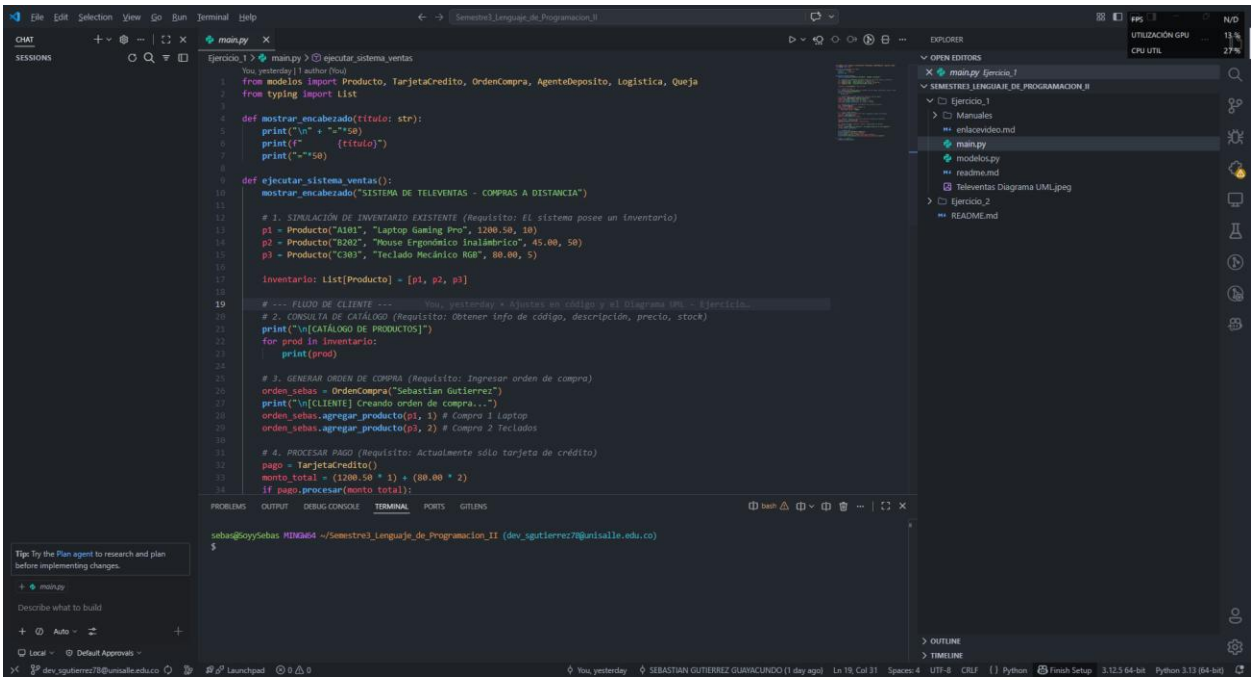
```
        self.cliente = cliente
        # Especificamos que el Dict tiene llaves String y valores de cualquier tipo (Any)
        self.items: List[Dict[str, Any]] = []
        self.estado = "PENDIENTE"

    def agregar_producto(self, producto: Producto, cantidad: int):
        if producto.cantidad >= cantidad:
            self.items.append({"prod": producto, "cant": cantidad})
            producto.cantidad -= cantidad
            print(f"[Orden] Agregado: {producto.descripcion} x{cantidad}")
        else:
            print(f"[Error] Stock insuficiente para {producto.descripcion}")

    def cancelar(self):
        for item in self.items:
            # Ahora Pylance sabe que 'item' es un Dict y puede acceder a sus llaves
            item["prod"].cantidad += item["cant"]
        self.estado = "CANCELADA"
        print(f"[Orden] Orden de {self.cliente} cancelada. Stock devuelto.")

class AgenteDeposito:
    def armar_pedido(self, orden: OrdenCompra):
        if orden.estado == "PENDIENTE":
            print(f"[Depósito] Armando y empaquetando pedido para {orden.cliente}...")
            orden.estado = "ARMADA"

class Logistica:
    @staticmethod
    def despachar(orden: OrdenCompra, empresa_transporte: str):
        if orden.estado == "ARMADA":
            print(f"[Logística] Pedido enviado vía {empresa_transporte}.")
            orden.estado = "ENVIADA"

class Queja:
    def __init__(self, cliente: str, motivo: str):
        self.cliente = cliente
        self.motivo = motivo

    def remitir_a_gerencia(self):
        print(f"[GERENCIA] Reclamo recibido de {self.cliente}: {self.motivo}")
```

6. Código en Visual Studio Code

main.py

Este archivo funciona como el controlador. Su función es:

1. **Instanciar objetos:** Crea los productos y usuarios iniciales.
2. **Ejecutar el Flujo:** Coordina la secuencia lógica (Venta -> Pago -> Empaque -> Envío).
3. **Manejo de Estados:** Es el encargado de verificar si el pago fue exitoso para actualizar la variable mi_orden.estado, disparando así el resto de los procesos del depósito.



modelos.py

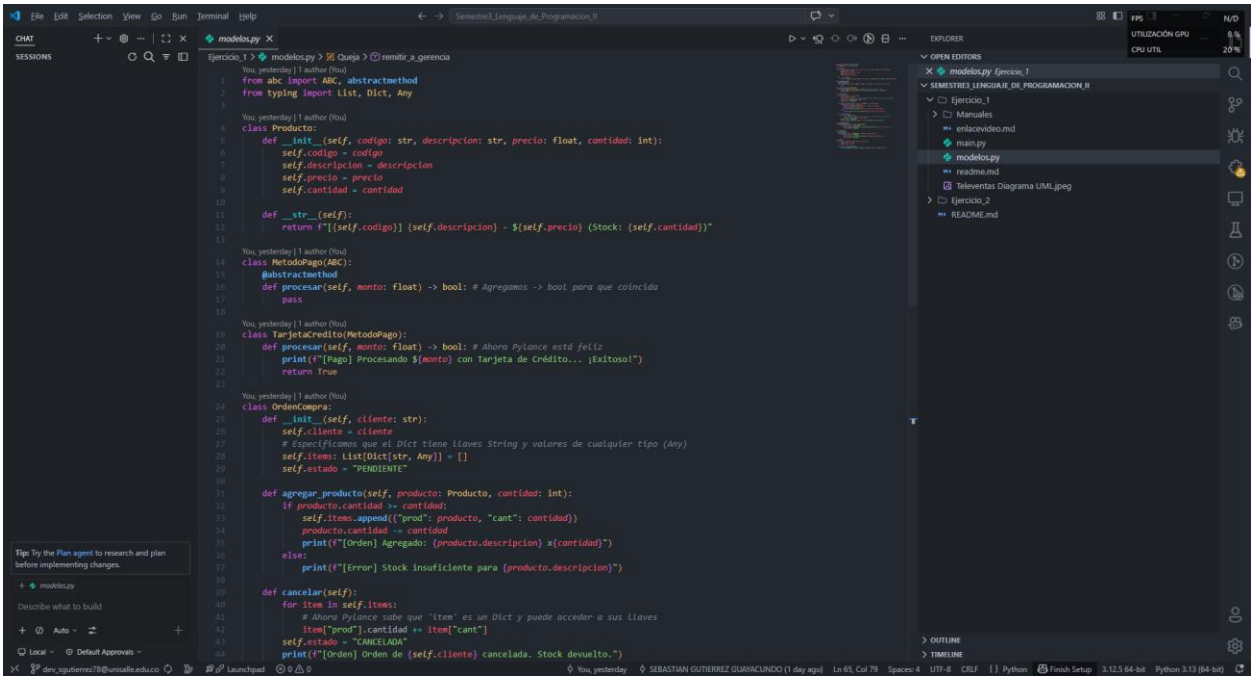
Este archivo funciona como el "motor" del sistema. Se construyó utilizando los siguientes pilares de la POO:

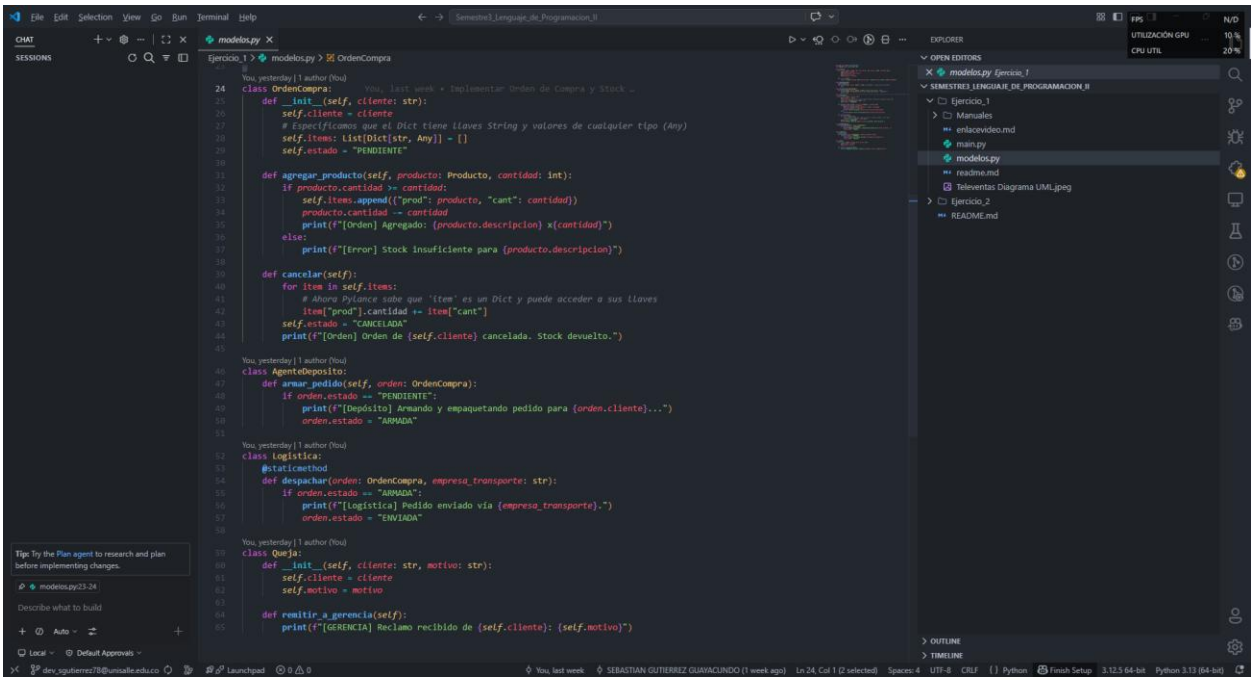
- **Clases de Entidad (Producto, Queja):** Almacenan los datos básicos y su representación textual.

- **Abstracción (MetodoPago):** Se definió una clase abstracta para que el sistema de cobro sea genérico. Esto permite que el software sea escalable; hoy usamos tarjeta de crédito, pero se pueden añadir otros medios sin modificar el código de la orden.

- **Lógica de Negocio (OrdenCompra):** Es la clase principal. Controla el flujo de estados y garantiza la integridad del inventario. Por ejemplo, el método agregar_item contiene la lógica para descontar stock automáticamente, y cancelar_orden contiene el algoritmo para devolverlo al sistema.

- **Servicios Especializados (AgenteDeposito, Logistica):** Clases que actúan como departamentos del mundo real. Logistica usa un método estático (@staticmethod) porque su función es de servicio delegado y no requiere guardar datos internos del transporte.





7. Prueba de Escritorio

Inicialización de Objetos (Memoria/Inventario)

En esta etapa se crean las instancias de Producto en la memoria.

Objeto	ID	Descripción	Precio
p1	A101	Laptop Gaming Pro	1200.50
p2	B202	Mouse Ergonómico	45.00
p3	C303	Teclado Mecánico	80.00

Flujo del Cliente (Orden de Compra)

Paso A: Creación de la Orden

- **Acción:** Se instancia orden_sebas = OrdenCompra("Sebastian Gutierrez").
- **Estado inicial:** orden_sebas.estado = "PENDIENTE" (asumido).

Paso B: Agregar Productos

1. orden_sebas.agregar_producto(p1, 1):
    - La orden registra 1 Laptop.
    - *Nota:* Dependiendo de la implementación, el stock de p1 podría bajar a **9** aquí o al procesar el pago.
2. orden_sebas.agregar_producto(p3, 2):
    - La orden registra 2 Teclados.
    - *Nota:* El stock de p3 podría bajar a **3**.

Paso C: Procesar Pago

- **Cálculo:** $monto_total = (1200.50 \times 1) + (80.00 \times 2) = 1360.50$.
- **Acción:** pago.procesar(1360.50) devuelve True.
- **Cambio de Estado:** orden_sebas.estado cambia de "PENDIENTE" a **"PAGADA"**.

Flujo Interno (Logística y Operaciones)

Paso	Actor	Acción	Resultado esperado
5	AgenteDeposito	armar_pedido(orden_sebas)	Cambia estado de la orden a "EMPAQUETADO" o "LISTO".
6	Logistica	despachar(orden_sebas, "Servientrega")	Cambia estado a "ENVIADO" o "DESPACHADO".
7	Queja	remitir_a_gerencia()	Imprime notificación de alerta para el gerente.

Salida en Consola

Al finalizar la ejecución, los valores finales que verías impresos serían:

- Cliente: Sebastian Gutierrez
- Estado Final de la Orden: "PAGADA" (o "DESPACHADA", si los métodos de logística modifican este atributo).
- Stock Actualizado de Laptop Gaming Pro: 9 unidades (suponiendo que agregar_producto o armar_pedido resta el stock).

Resultado:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

sebas@SoyySebas MINGW64 ~/Semestre3_Lenguaje_de_Programacion_II (dev_sgutierrez78@unisalle.edu.co)
$ C:/Users/sebas/AppData/Local/Programs/Python/Python312/python.exe c:/Users/sebas/Semestre3_Lenguaje_de_Programacion_II/Ejercicio_1/mai
n.py

==================================================
       SISTEMA DE TELEVENTAS - COMPRAS A DISTANCIA
==================================================

[CATÁLOGO DE PRODUCTOS]
[A101] Laptop Gaming Pro - $1200.5 (Stock: 10)
[B202] Mouse Ergonómico inalámbrico - $45.0 (Stock: 50)
[C303] Teclado Mecánico RGB - $80.0 (Stock: 5)

[CLIENTE] Creando orden de compra...
[Orden] Agregado: Laptop Gaming Pro x1
[Orden] Agregado: Teclado Mecánico RGB x2
[Pago] Procesando $1360.5 con Tarjeta de Crédito... ¡Exitoso!

------------------------------
[GERENCIA] Reclamo recibido de Sebastian Gutierrez: El paquete llegó con la caja golpeada.

==================================================
       RESUMEN DE OPERACIÓN
==================================================
Cliente: Sebastian Gutierrez
Estado Final de la Orden: PAGADA
Stock Actualizado de Laptop Gaming Pro: 9 unidades

sebas@SoyySebas MINGW64 ~/Semestre3_Lenguaje_de_Programacion_II (dev_sgutierrez78@unisalle.edu.co)
$
```

Tras la ejecución de la simulación y el análisis de la trazabilidad en consola, se pueden extraer las siguientes conclusiones:

- **Integridad del Proceso de Negocio:** El sistema garantiza un flujo secuencial lógico. Como se observa en el log, la Logística solo se activa después de que el Depósito confirma el empaque, y este a su vez solo actúa tras una Transacción exitosa. Esto demuestra un control efectivo de los estados del objeto OrdenCompra.

- **Gestión Eficiente de Inventario:** El mensaje [Orden] Agregado confirma que la comunicación entre la clase OrdenCompra y Producto es bidireccional, permitiendo la reserva inmediata del stock para evitar ventas en exceso (overselling).

- **Desacoplamiento de Soporte (Escalabilidad):** El sistema de Quejas funciona de manera independiente al flujo de despacho. El resultado muestra que, aunque el pedido ya fue entregado a la transportadora, el módulo de soporte puede capturar incidencias y remitirlas a la Gerencia sin afectar la base de datos de ventas o logística.

- **Eficacia de la Abstracción:** El uso de una clase abstracta para el pago permitió que el sistema procesara la Tarjeta de Crédito de forma transparente, validando que la arquitectura es flexible para integrar nuevos métodos de pago en futuras fases del proyecto.